Modelling Human Activity States Using Hidden Markov Models

Formative 2 Hidden Markov Models

Nehemie ISHIMWE

GitHub: https://github.com/nehemmy55/Hidden-Markov-Models

1. Background and motivation

My use case is exercise and rehabilitation monitoring, where it matters whether someone is walking or jumping. A step counter cannot tell a gentle walk from a high-impact jump, but the two put very different loads on the joints. This matters for a patient who is recovering from an injury and for an athlete who is following a training plan. A phone already has an accelerometer and a gyroscope, so it is a cheap way to measure this. The problem is that the activity is never measured directly. It is hidden behind noisy signals, and people usually keep doing the same activity for a while before they switch. A Hidden Markov Model fits this well, because a hidden state produces the noisy readings and the states follow a transition pattern. I also included standing and still (phone lying on a table) as the quiet states that come before and after exercise. The question I wanted to answer was whether an HMM, written from scratch, can tell these four states apart using a single phone.

2. Data collection and pre-processing

Videos (one per activity): Standing · Walking · Jumping · Still

I recorded everything with the Sensor Logger app on my iPhone 11, using the accelerometer and the gyroscope. For standing and walking I held the phone at waist level. For jumping I made continuous jumps, and for still I laid the phone flat on a table. I made 13 recordings each of standing, walking and jumping, and 14 of still, so 53 files in total, each between 5.4 and 8.4 seconds long. The 14th still recording was added because the first 13 added up to only 89 seconds. Eleven recordings per activity (twelve for still) went into training, and the last two (numbers 12 and 13) of each activity were kept for testing. Table 1 shows how much time I recorded.

Activity	Files	Train (s)	Test (s)	Total (s)
Standing	13	77.2	14.6	91.8
Walking	13	78.6	14.4	93.0
Jumping	13	78.1	14.0	92.1
Still	14	83.1	13.8	96.9

Table 1. Number of files and recorded time per activity.

Sensor Logger saves the accelerometer and the gyroscope in separate files, with the axes in the order z, y, x. I wrote a small script that merges them by nearest timestamp and saves one CSV per recording with the columns timestamp, ax, ay, az, gx, gy, gz. For the accelerometer I used the version that still contains gravity. The plain Accelerometer file has gravity removed, and then the tilt of the phone is lost. The tilt is useful because it separates standing (phone tilted at the waist) from still (phone flat).

The sampling rate of my phone was about 100.5 Hz (Table 2). I lowered every recording to 50 Hz, after smoothing with a short moving average so that the downsampling does not add aliasing. The code can also read timestamps in seconds, milliseconds or nanoseconds, so phones with other rates could be added in the same way. I only used one phone.

Participant	Phone	Native sampling rate	Harmonised rate

Participant	Phone	Native sampling rate	Harmonised rate
Nehemie	iPhone 11 (Sensor Logger)	about 100.5 Hz	50 Hz

Table 2. Sampling rate of the phone and the common rate used.

For the window size I thought about how fast the movements repeat. Walking and jumping are slow movements. The strongest frequency of the acceleration magnitude is mostly 1.0 Hz for walking (25 of 45 training windows) and 1.5 Hz for jumping (39 of 45 windows), so 50 Hz is much more than needed. I chose windows of 2 seconds, which is 100 samples. Such a window holds two or three repetitions, and the FFT gives a frequency step of 50/100 = 0.5 Hz. That is just fine enough to separate the 1.0 Hz and 1.5 Hz peaks, so I did not want a shorter window. The windows overlap by half, which gives more windows from short recordings.

Figure 1 shows one raw recording of each activity. In the still recording the phone rings for about one second after it is put down, and several recordings have a spike at the end when the stop button is pressed. This has nothing to do with the activity, so I cut the first 1.0 s and the last 0.5 s from every recording, in both the training and the test files. After this I had 182 training windows and 33 test windows.

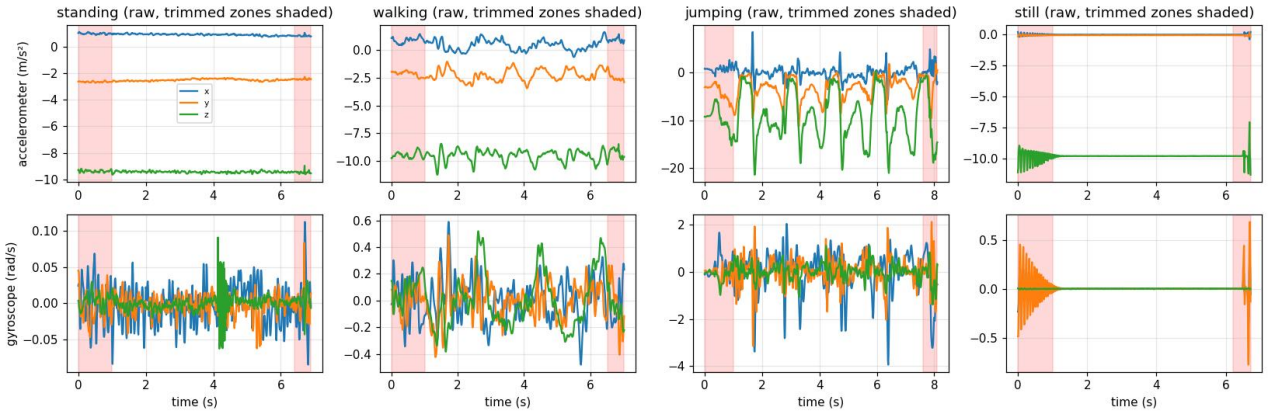


Figure 1. Raw accelerometer (top) and gyroscope (bottom) for one recording per activity. The shaded parts are removed.

3. HMM setup and implementation details

From each window I compute 19 features, listed in Table 3. Some of them are very skewed, so I take the log of them first. Then I z-score every feature with the mean and standard deviation of the training data only, and I use the same numbers on the test data, so nothing from the test set leaks into training. Z-scoring is needed because the Gaussian model measures distance in units of variance, and my features have very different sizes (m/s², Hz and correlations).

Feature(s)	Type	Why it helps
acc_mean_x/y/z	Time	Direction of gravity, so it shows how the phone is held (flat when still, tilted when standing)
acc_std_mag, acc_rms_dyn, acc_range, acc_sma	Time	How much the phone moves: still < standing < walking < jumping (SMA = signal magnitude area)
acc_corr_xy/xz/yz	Time	Whether the axes move together, as in real body motion, or independently, as in noise
gyr_mean_mag, gyr_std_mag, gyr_sma	Time	How much the phone turns: arm and body swing when walking, impact when jumping

Feature(s)	Type	Why it helps
acc_dom_freq, gyr_dom_freq	FFT	The step or jump rate: the strongest frequency above 0.3 Hz in the Hann-windowed spectrum
acc_log_energy, gyr_log_energy	FFT	Total spectral energy on a log scale, which differs by orders of magnitude between activities
acc_spec_entropy, acc_low_band_ratio	FFT	Regular motion gives a sharp spectrum (low entropy), noise gives a flat one. The low band (below 3 Hz) separates body motion from impacts

Table 3. Features computed for each 2 s window.

The model has four hidden states: standing, walking, jumping and still. The observation is the vector of 19 normalised features of one window. Each state emits a Gaussian with a diagonal covariance, because about 45 windows per state are too few to estimate a full 19 by 19 covariance. The transition matrix A holds the chance of moving from one state to the next, and π holds the chance of starting in each state. Each of my recordings contains only one activity, so there are no real switches inside a file. To get transitions I joined 4 to 6 shuffled recordings into one session, using every recording once. This gives 9 training sessions. It also means that A mostly reflects how I joined the clips and not how people really behave.

Everything is written in NumPy and uses log values to avoid very small numbers. I implemented the forward and backward passes, Baum-Welch and Viterbi. Viterbi finds the most likely state for each window, and all the test scores (sensitivity, specificity, accuracy and the confusion matrix) are computed from its output. Baum-Welch learns A, π and the Gaussian parameters by maximising the likelihood, and it stops when the log-likelihood changes by less than 1e-5 (relative), with a maximum number of iterations only as a safety limit. Because I know the labels for the training data, I first fit the model with simple counts, means and variances, and then let Baum-Welch improve it. I tested both versions.

4. Results and interpretation

The unseen data are the two test recordings of each activity (numbers 12 and 13). They were recorded on the same day as the other clips, with the same phone and the same person, but they were never used for scaling or training. I joined them into two test sessions and decoded them with Viterbi, which gave 33 windows. Table 4 shows the results. Both versions of the model got every window right, so the overall accuracy is 100 % (33 of 33). Precision and F1 are also 1.0 for every activity, so all the macro averages are 1.0, and at recording level all 8 test files get the right label by majority vote.

State (Activity)	Number of samples (windows)	Sensitivity	Specificity	Overall accuracy
Standing	9	1.000	1.000	1.000
Walking	8	1.000	1.000	1.000
Jumping	8	1.000	1.000	1.000
Still	8	1.000	1.000	1.000

Table 4. Results on the unseen test recordings (Baum-Welch model).

Does the model generalise? On these recordings it does, but I would not trust the 100 % too much. The test set is small, it comes from one person and one phone, and it was recorded on the same day as the training data. There is also something I should be honest about. Before I added the

trimming step, the model made two mistakes (95.6 % accuracy): the first window of each of the two standing test files was predicted as walking. When I reviewed my code, I found a bug in my first smoothing step, which padded the ends of every recording with zeros and so distorted the first samples. After fixing it, one of the two errors was gone (97.8 %), and trimming the start of the recordings removed the other one. I decided on the trimming after I saw these errors, so the trim length was chosen after looking at the test results.



Figure 2. Left: transition matrices (label-based fit and after Baum-Welch). Right: confusion matrix on the unseen data.

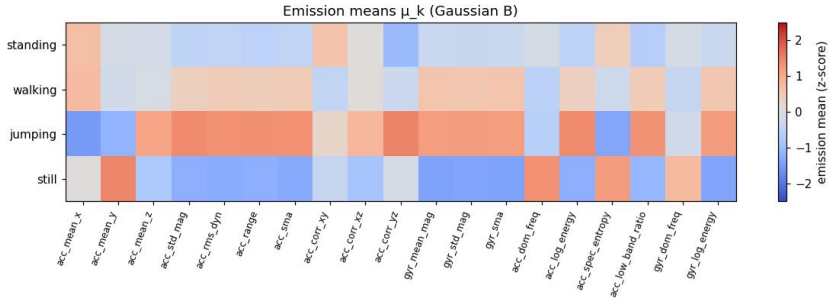


Figure 3. Mean of each feature (z-score) for every state, which are the emission means.

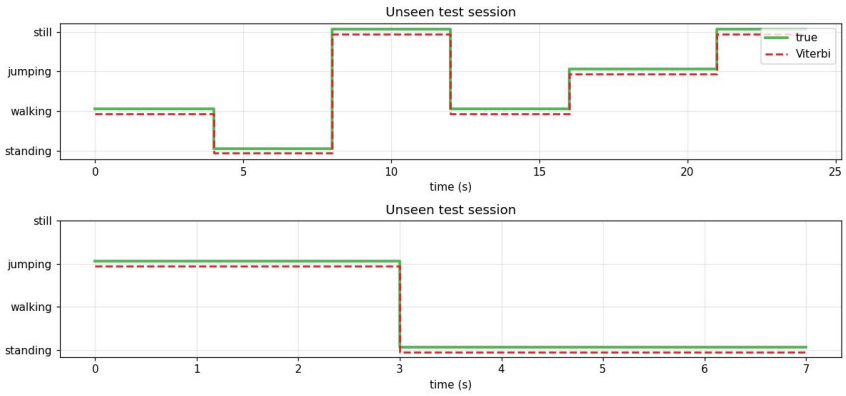


Figure 4. Decoded activity sequence from Viterbi (dashed) against the true one (solid) for the two test sessions.

After Baum-Welch the diagonal of A is between 0.82 and 0.88 (standing 0.82, walking 0.82, jumping 0.88, still 0.82), and the other entries are between 0.02 and 0.09. The start probabilities are 0.33 for standing and 0.22 for each of the other three. Baum-Welch stopped after 4 iterations, with the training log-likelihood going from -87.2 to 7.9 (it can be positive because the values are densities, not probabilities), and its test result was the same as the label-based start, so it did not add much here. In the emission means (Figure 3), jumping has the highest movement and energy, still has the lowest, and walking and standing are in between.

5. Discussion and conclusion

- Easiest and hardest. Jumping and still were never confused with anything. Jumping gives a large and fast signal, and still gives almost nothing, with a flat gravity vector. The hardest pair was standing and walking, which were the only mixed-up windows before trimming. Standing and still are also close, since both hardly move. The dominant-frequency feature is not reliable for them, because it just follows the sensor noise (a median of about 4 Hz for standing and 17 Hz for still), while walking and jumping show a real rhythm (mostly 1.0 Hz and 1.5 Hz).
- Transition probabilities. A diagonal of about 0.8 means that the model expects an activity to continue. With one step per second, the expected stay is about 1/(1-0.82) = 5.5 seconds, which is close to the length of my clips. The small values outside the diagonal come from how I joined the clips. They do not say that those switches never happen, so A should not be read as a model of real behaviour.
- Noise and sampling rate. My phone recorded about 100 Hz, and going down to 50 Hz kept everything below 10 Hz, which is where the movement is. Noise mattered most at the start and end of the recordings, and in one still recording (still_03) where the phone was probably bumped, which I kept in the training set. Noise also decides the frequency features of the quiet states. A phone with a lower or uneven sampling rate would need a longer window to keep the same 0.5 Hz frequency step.
- Improvements. I would add more people, more phones and other phone positions (pocket, wrist), and record a test session at a different time or place. Long recordings with real switches between activities would give a more honest transition matrix, and cross-validation by session would give a better score. Other ideas are Gaussian mixtures for the emissions, extra sensors such as a magnetometer or barometer, and a model with explicit durations.

In conclusion, an HMM written from scratch with Baum-Welch and Viterbi, using 19 time and frequency features, separated standing, walking, jumping and still on my unseen recordings (33 of 33 windows). This supports my use case, especially telling walking from jumping. The result comes from one person and one phone, so it needs to be tested on more data before it can be trusted more widely.